

CMakeLists.txt

目录

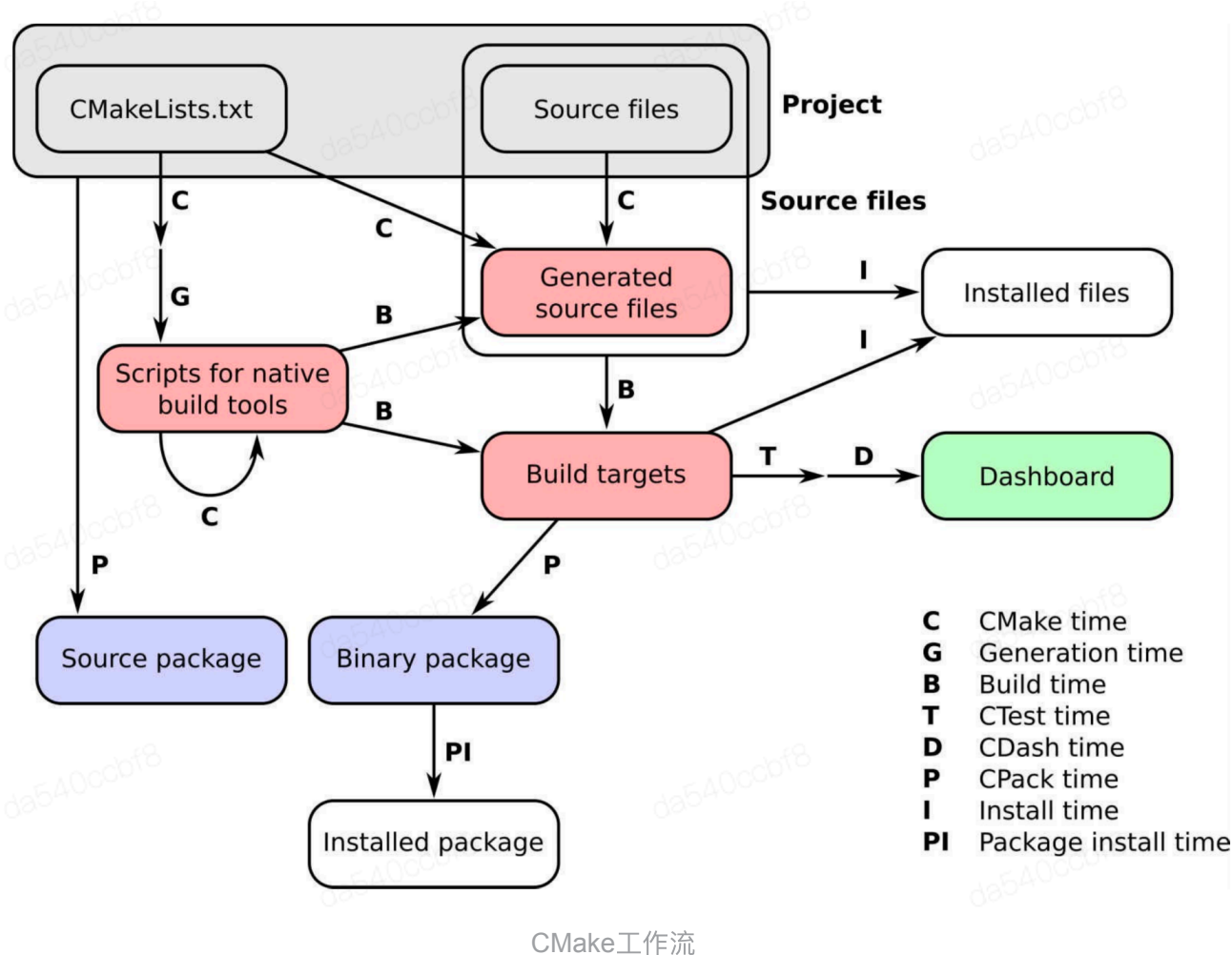
- **CMake** 工作流
 - 配置和构建时的自定义行为
 - 生成器表达式
 - 生成源代码
 - 1. 在配置时生成代码
- 构建项目
- **Tips**

CMake 工作流

先简单回顾一下，与 CMake 工作流程相关的时序：

1. **CMake 时或构建时**：CMake 正在运行，并处理项目中的 `CMakeLists.txt` 文件。
2. **生成时**：生成构建工具(如 Makefile 或 Visual Studio 项目文件)。
3. **构建时**：由 CMake 生成相应平台的原生构建脚本，在脚本中调用原生工具构建。此时，将调用编译器在特定的构建目录中构建目标(可执行文件和库)。
4. **CTest 时或测试时**：运行测试套件以检查目标是否按预期执行。
5. **CDash 时或报告时**：当测试结果上传到仪表板上，与其他开发人员共享测试报告。
6. **安装时**：当目标、源文件、可执行程序 and 库，从构建目录安装到相应位置。
7. **CPack 时或打包时**：将项目打包用以分发时，可以是源码，也可以是二进制。
8. **包安装时**：新生成的包在系统范围内安装。

完整的工作流程和对应的时序，如下图所示：



CMake工作流

i 关于Generated source files设计C和B阶段的解释：
生成的源文件可能依赖于项目中的source files，如何操作取决于CMakeLists中声明的指令，而由于这些指令依赖于生成器和目标平台，因此，生成的源文件依赖于B阶段，且只能在B阶段生成其对应的构建目标。

配置和构建时的自定义行为

如下展示了在配置和构建时常见的自定义行为：

- **execute_process**，从CMake中执行任意进程，并检索它们的输出。
- **add_custom_target**，创建执行自定义命令的目标。
- **add_custom_command**，指定必须执行的命令，以生成文件或在其他目标的特定生成事件中生成。

生成器表达式

CMake分两个阶段生成项目的构建系统：配置阶段(解析 **CMakeLists.txt**)和生成阶段(实际生成构建环境)。生成器表达式在第二阶段进行计算，可以使用仅在生成时才能知道的信息来调整构建系统。生成器表达式在交叉编译时特别有用，一些可用的信息只有解析 **CMakeLists.txt** 之后，或在多配置项目后获取。

CMake提供了三种类型的生成器表达式，完整列表链接为：[🔗](#)：

- **逻辑表达式**，基本模式为 `$<condition:outcome>`。基本条件为0表示false, 1表示true，但是只要使用了正确的关键字，任何布尔值都可以作为条件变量。
- **信息表达式**，基本模式为 `$<information>` 或 `$<information:input>`。这些表达式对一些构建系统信息求值，例如：包含目录、目标属性等等。这些表达式的输入参数可能是目标的名称，比如表达式 `$<TARGET_PROPERTY:tgt,prop>`，将获得的信息是tgt目标上的prop属性。
- **输出表达式**，基本模式为 `$<operation>` 或 `$<operation:input>`。这些表达式可能基于一些输入参数，生成一个输出。它们的输出可以直接在CMake命令中使用，也可以与其他生成器表达式组合使用。例如，
`IS<JOIN:$<TARGET_PROPERTY:INCLUDE_DIRECTORIES>, -I>` 将生成一个字符串，其中包含正在处理的目标的包含目录，每个目录的前缀由 `-I` 表示。

生成源代码

- 大多数项目，使用版本控制跟踪源码。源代码通常作为构建系统的输入，将其转换为o文件、库或可执行程序。但是在某些情况下，我们在配置或构建步骤时会生成源代码，根据配置步骤中收集的信息，对源代码进行微调。另一个常用的方式，是记录有关配置或编译的信息，以保证代码行为可重现性。

1. 在配置时生成代码

在配置时，CMake通过解析CMakeLists文件可以检测目标平台的操作系统和可用库信息。基于这些信息，我们可以定制构建的源代码。

构建项目

- CMake宏--类似内联函数，可修改外部变量。
CMake函数--一般不产生输出，可视作黑盒。
- `add_subdirectory()`命令会调用对应子目录的CMakeLists.txt
- CMake为所有目标定义了`interface_include_directory`属性，如果target依赖的头文件所在目录不在其中，则需要手动使用`target_include_directory()`添加。

Tips

- ❗ CMake是一个构建系统生成器。可以为当前平台支持的不同的构建系统生成其对应的构建指令。因此CMake内部的实现中包含多种生成器。在Unix/Linux平台中，`cmake`默认调用Unix Makefiles生成器。

执行`cmake`命令对项目源代码进行生成构建文件操作后，可以在`build`目录下执行如下命令来查询可构建的目标。

- ❗ `cmake --build . --target help`

- ❗ 对于源代码的汇集，对于简单的项目，我们通常使用`add_library`来集合源码，对于大项目而言，生成目标往往需要更多的源码，因此常使用 `target_sources` 来汇集源码。

- ❗ CMake有适当的机制，通过包含模块来扩展其语法和功能，这些模块要么是CMake自带的，要么是定制的。
可以通过 `cmake --help-module <name-of-module>` 命令查看具体CMake自带的模块功能说明。

- ❗ 通过 `cmake --help-command <command-name>` 来查看CMake中特定命令的文档。

- ❗ CMake可以配置构建类型，例如：Debug、Release等。
控制生成构建系统使用的配置变量是 `CMAKE_BUILD_TYPE` 。该变量默认为空。

- ❗ `Find<package>.cmake` 模块试图提供统一的检测接口，可以在终端中直接浏览文档，例如可使用 `cmake --help-module FindPythonInterp` 查看Python解释器的路径。

- ❗ 使用以下CLI开关，可以从CTest获得更详细的测试输出：
 - `--output-on-failure` :将测试程序生成的任何内容打印到屏幕上，以免测试失败。
 - `-v` :将启用测试的详细输出。
 - `-vv` :启用更详细的输出。

CTest提供了一个非常方便快捷的方式，可以重新运行以前失败的测试；要使用的CLI开关是 `--rerun-failed` ，在调试期间非常有用。

